Chapter 10. LINQ

Language Integrated Query (LINQ) is a powerful collection of C# language features for working with sets of information. It is useful in any application that needs to work with multiple pieces of data (i.e., almost any application). Although one of its original goals was to provide straightforward access to relational databases, LINQ is applicable to many kinds of information. For example, it can also be used with in-memory object models, HTTP-based information services, JSON, and XML documents. And as we'll see in Chapter 11, it can work with live streams of data too.

LINQ is not a single feature. It relies on several language elements that work together. The most conspicuous LINQ-related language feature is the *query expression*, a form of expression that loosely resembles a database query but that can be used to perform queries against any supported source, including plain old objects. As you'll see, query expressions rely heavily on some other language features such as lambdas, extension methods, and expression object models.

Language support is only half the story. LINQ needs class libraries to implement a set of querying primitives called *LINQ operators*. Each different kind of data requires its own implementation, and a set of operators for any particular type of information is referred to as a *LINQ provider*. (These can also be used from Visual Basic and F#, by the way, because those languages support LINQ too.) Microsoft supplies several providers, some built into the runtime libraries and some available as separate NuGet packages. There is a provider for Entity Framework Core (EF Core) for example, an object/relational mapping system for working with databases. The Cosmos DB cloud database (a feature of Microsoft Azure) offers a LINQ provider. And the Reactive Extensions for .NET (Rx) described in Chapter 11 provide LINQ support for live streams of data. In short,

LINQ is a widely supported idiom in .NET, and it's extensible, so you will also find open source and other third-party providers.

Most of the examples in this chapter use LINQ to Objects. This is partly because it avoids cluttering the examples with extraneous details such as database or service connections, but there's a more important reason. LINQ's introduction in 2007 significantly changed the way I write C#, and that's entirely because of LINQ to Objects. Although LINQ's query syntax makes it look like it's primarily a data access technology, I have found it to be far more valuable than that. Having LINQ's services available on any collection of objects makes it useful in every part of your code.

Query Expressions

The most visible feature of LINQ is the query expression syntax. It's not the most important—as we'll see later, it's entirely possible to use LINQ productively without ever writing a query expression. However, it's a very natural syntax for many kinds of queries.

At first glance, a query expression loosely resembles a relational database query, but the syntax works with any LINQ provider. Example 10-1 shows a query expression that uses LINQ to Objects to search for certain `CultureInfo` objects. (A `CultureInfo` object provides a set of culture-specific information, such as the symbol used for the local currency, what language is spoken, and so on. Some systems call this a *locale*.) This particular query looks at the character that denotes what would, in English, be called the decimal point. Many countries actually use a comma instead of a period, and in those countries, 100,000 would mean the number 100 written out to three decimal places; in English-speaking cultures, we would normally write this as 100.000. The query expression searches all the cultures known to the system and returns those that use a comma as the decimal separator.

Example 10-1. A LINQ query expression

```
IEnumerable<CultureInfo> commaCultures =
```

```
    from culture in CultureInfo.GetCultures(CultureTypes.AllCultures)
    where culture.NumberFormat.NumberDecimalSeparator == ","
    select culture;

foreach (CultureInfo culture in commaCultures)
{
    Console.WriteLine(culture.Name);
}
```

The `foreach` loop in this example shows the results of the query. The output will vary according to the language support installed on the system you run it on. On my system, this lists the names of 366 cultures, indicating that slightly under half of the 869 available cultures use a comma, not a decimal point. Of course, I could easily have achieved this without using LINQ. Example 10-2 will produce the same results.

Example 10-2. The non-LINQ equivalent

```
CultureInfo[] allCultures = CultureInfo.GetCultures(CultureTypes.AllCultures);
foreach (CultureInfo culture in allCultures)
{
    if (culture.NumberFormat.NumberDecimalSeparator == ",")
    {
        Console.WriteLine(culture.Name);
    }
}
```

Both examples have eight nonblank lines of code, although if you ignore lines that contain only braces, Example 10-2 contains just four, two fewer than Example 10-1. Then again, if we count statements, the LINQ example has just three, compared to four in the loop-based example. So it's difficult to argue convincingly that either approach is simpler than the other.

However, Example 10-1 has a significant advantage: the code that decides which items to choose is well separated from the code that decides what to do with those items. Example 10-2 intermingles these two concerns: the code that picks the objects is half outside and half inside the loop.

Another difference is that Example 10-1 has a more declarative style: it focuses on what we want, not how to get it. The query expression de-

scribes the items we'd like, without mandating that this be achieved in any particular way. For this very simple example, that doesn't matter much, but for more complex examples, and particularly when using a LINQ provider for database access, it can be very useful to allow the provider a free hand in deciding exactly how to perform the query. Example 10-2's approach of iterating over everything in a `foreach` loop and picking the item it wants would be a bad idea if we were talking to a database—you generally want to let the server do this sort of filtering work.

The query in Example 10-1 has three parts. All query expressions are required to begin with a `from` clause, which specifies the source of the query. In this case, the source is an array of type `CultureInfo[]`, returned by the `CultureInfo` class's `GetCultures` method. As well as defining the source for the query, the `from` clause contains a name, `culture`. This is called the *range variable*, and we can use it in the rest of the query to represent a single item from the source. Clauses can run many times—the `where` clause in Example 10-1 runs once for every item in the collection, so the range variable will have a different value each time. This is reminiscent of the iteration variable in a `foreach` loop. In fact, the overall structure of the `from` clause is similar—we have the variable that will represent an item from a collection, then the `in` keyword, then the source for which that variable will represent individual items. Just as a `foreach` loop's iteration variable is in scope only inside the loop, the range variable `culture` is meaningful only inside this query expression.

NOTE

Although analogies with `foreach` can be helpful for understanding the intent of LINQ queries, you shouldn't take this too literally. For example, not all providers directly execute the expressions in a query. Some LINQ providers convert query expressions into database queries, in which case the C# code in the various expressions inside the query does not run in any conventional sense. So, although it is true to say that the range variable represents a single value from the source, it's not always true to say that clauses will execute once for every item they process, with the range value taking that item's value. It happens to be true for Example 10-1 because it uses LINQ to Objects, but it's not so for all providers.

The second part of the query in Example 10-1 is a `where` clause. This clause is optional, or if you want, you can have several in one query. A `where` clause filters the results, and the one in this example states that I want only the `CultureInfo` objects with a `NumberFormat` that indicates that the decimal separator is a comma.

The final part of the query is a `select` clause. All query expressions end with either a `select` clause or a `group` clause. This determines the final output of the query. This example indicates that we want each `CultureInfo` object that was not filtered out by the query. The `foreach` loop in Example 10-1 that shows the results of the query uses only the `Name` property, so I could have written a query that extracted only that. As Example 10-3 shows, if I do this, I also need to change the loop, because the resulting query now produces strings instead of `CultureInfo` objects.

Example 10-3. Extracting just one property in a query

```
IEnumerable<string> commaCultures =
    from culture in CultureInfo.GetCultures(CultureTypes.AllCultures)
    where culture.NumberFormat.NumberDecimalSeparator == ","
    select culture.Name;

foreach (string cultureName in commaCultures)
{
    Console.WriteLine(cultureName);
}
```

This raises a question: In general, what type do query expressions have? In Example 10-1, `commaCultures` is an `IEnumerable<CultureInfo>`; in Example 10-3, it's an `IEnumerable<string>`. The output item type is determined by the final clause of the query—the `select` or, in some cases, the `group` clause. However, not all query expressions result in an `IEnumerable<T>`. It depends on which LINQ provider you use—I've ended up with `IEnumerable<T>` because I'm using LINQ to Objects.

NOTE

It's common to use the `var` keyword when declaring variables that hold LINQ queries. This is necessary if a `select` clause produces instances of an anonymous type, because there is no way to write the name of the resulting query's type. Even if anonymous types are not involved, `var` is still widely used, and there are two reasons. One is just a matter of consistency: some people feel that because you have to use `var` for some LINQ queries, you should use it for all of them. Another argument is that LINQ query types often have verbose and ugly names, and `var` results in less cluttered code. In this chapter I have used `var` where necessary.

How did C# know that I wanted to use LINQ to Objects? It's because I used an array as the source in the `from` clause. More generally, LINQ to Objects will be used when you specify any `IEnumerable<T>` as the source, unless a more specialized provider is available. However, this doesn't really explain how C# discovers the existence of providers in the first place and how it chooses between them. To understand that, you need to know what the compiler does with a query expression.

How Query Expressions Expand

The compiler converts all query expressions into one or more method calls. Once it has done that, the LINQ provider is selected through exactly the same mechanisms that C# uses for any other method call. The compiler does not have any built-in concept of what constitutes a LINQ provider. It just relies on convention. Example 10-4 shows what the compiler does with the query expression in Example 10-3.

Example 10-4. The effect of a query expression

```
IEnumerable<string> commaCultures =
    CultureInfo.GetCultures(CultureTypes.AllCultures)
    .Where(culture => culture.NumberFormat.NumberDecimalSeparator == ",")
    .Select(culture => culture.Name);
```

The `Where` and `Select` methods are examples of LINQ operators. A LINQ operator is nothing more than a method that conforms to one of the stan-

dard patterns. I'll describe these patterns later, in "Standard LINQ Operators".

The code in Example 10-4 is all one statement, and I'm chaining method calls together—I call the `Where` method on the return value of `GetCultures`, and I call the `Select` method on the return value of `Where`. The formatting looks a little peculiar, but it's too long to go on one line; and, even though it's not terribly elegant, I prefer to put the `.` at the start of the line when splitting chained calls across multiple lines, because it makes it much easier to see that each new line continues from where the last one left off. Leaving the period at the end of the preceding line looks neater but also makes it much easier to misread the code.

The compiler has turned the `where` and `select` clauses' expressions into lambdas. Notice that the range variable ends up as a parameter in each lambda. This is one example of why you should not take the analogy between query expressions and `foreach` loops too literally. Unlike a `foreach` iteration variable, the range variable does not exist as a single conventional variable. In the query, it is just an identifier that represents an item from the source, and in expanding the query into method calls, C# may end up creating multiple real variables for a single range variable, like it has with the arguments for the two separate lambdas here.

All query expressions boil down to this sort of thing—chained method calls with lambdas. (This is why we don't strictly need the query expression syntax—you could write any query using method calls instead.) Some are more complex than others. The expression in Example 10-1 ends up with a simpler structure despite looking almost identical to Example 10-3. Example 10-5 shows how it expands. It turns out that when a query's `select` clause just passes the range variable straight through, the compiler interprets that as meaning that we want to pass the results of the preceding clause straight through without further processing, so it doesn't add a call to `Select`. (There is one exception to this: if you write a query expression that contains nothing but a `from` and a `select` clause, it will generate a call to `Select` even if the `select` clause is trivial.)

Example 10-5. How trivial `select` clauses expand

```
IEnumerable<CultureInfo> commaCultures =
    CultureInfo.GetCultures(CultureTypes.AllCultures)
    .Where(culture => culture.NumberFormat.NumberDecimalSeparator == ",");
```

The compiler has to work harder if you introduce multiple variables within the query's scope. You can do this with a `let` clause. Example 10-6 performs the same job as Example 10-3, but I've introduced a new variable called `numFormat` to refer to the number format. This makes my `where` clause shorter and easier to read, and in a more complex query that needed to refer to that format object multiple times, this technique could remove a lot of clutter.

Example 10-6. Query with a `let` clause

```
IEnumerable<string> commaCultures =
    from culture in CultureInfo.GetCultures(CultureTypes.AllCultures)
    let numFormat = culture.NumberFormat
    where numFormat.NumberDecimalSeparator == ","
    select culture.Name;
```

When you write a query that introduces additional variables like this, the compiler automatically generates an anonymous type with a property for each of the variables so that it can make them all available at every stage. To get the same effect with ordinary method calls, we'd need to do something similar, as Example 10-7 shows.

Example 10-7. How multivariable query expressions expand (approximately)

```
IEnumerable<string> commaCultures =
    CultureInfo.GetCultures(CultureTypes.AllCultures)
    .Select(culture => new { culture, numFormat = culture.NumberFormat })
    .Where(vars => vars.numFormat.NumberDecimalSeparator == ",")
    .Select(vars => vars.culture.Name);
```

No matter how simple or complex they are, query expressions are nothing more than a specialized syntax for method calls.

Deferred Evaluation

LINQ to Objects has been designed to work well with sequences like the one returned by the `Fibonacci` method in Example 10-8. That returns a never-ending sequence—it will keep providing numbers from the Fibonacci series for as long as the code keeps asking for them. I have used the `IEnumerable<BigInteger>` returned by this method as the source for a query expression.

Example 10-8. Query with an infinite source sequence

```
using System.Numerics;

static IEnumerable<BigInteger> Fibonacci()
{
    BigInteger n1 = 1;
    BigInteger n2 = 1;
    yield return n1;
    while (true)
    {
        yield return n2;
        BigInteger t = n1 + n2;
        n1 = n2;
        n2 = t;
    }
}

IEnumerable<BigInteger> evenFib = from n in Fibonacci()
                                  where n % 2 == 0
                                  select n;

foreach (BigInteger n in evenFib)
{
    Console.WriteLine(n);
}
```

This will use the `Where` extension method that LINQ to Objects provides for `IEnumerable<T>`. You could imagine an implementation of `Where` that iterates through its source collection, putting items that match the criteria into a `List<T>` that it returns once it has worked through the whole input. But if it worked that way, this program would never make it as far as displaying a single number. `Where` can never work its way through the whole input here because my Fibonacci enumerator is infinite.

In fact, Example 10-8 works perfectly—it produces a steady stream of output consisting of the Fibonacci numbers that are divisible by 2. This means it can't be attempting to perform all of the filtering when we call `Where`. Instead, its `Where` method returns an `IEnumerable<T>` that filters items on demand. It won't try to fetch anything from the input sequence until something asks for a value, at which point it will start retrieving one value after another from the source until the filter delegate says that a match has been found. It then produces that and doesn't try to retrieve anything more from the source until it is asked for the next item. Example 10-9 shows how you could implement this behavior by taking advantage of C#'s `yield return` feature.

Example 10-9. A custom deferred `Where` operator

```
public static class CustomDeferredLinqProvider
{
    public static IEnumerable<T> Where<T>(this IEnumerable<T> src,
                                          Func<T, bool> filter)
    {
        foreach (T item in src)
        {
            if (filter(item))
            {
                yield return item;
            }
        }
    }
}
```

The real LINQ to Objects implementation of `Where` is somewhat more complex. It detects certain special cases, such as arrays and lists, and it handles them in a way that is slightly more efficient than the general-purpose implementation that it falls back to for other types. However, the principle is the same for `Where` and all of the other operators: these methods do not perform the specified work. Instead, they return objects that will perform the work on demand. It's only when you attempt to retrieve the results of a query that anything really happens. This is called *deferred evaluation*, or sometimes *lazy evaluation*.

Deferred evaluation has the benefit of not doing work until you need it, and it makes it possible to work with infinite sequences. However, it also has disadvantages. You may need to be careful to avoid evaluating queries multiple times. Example 10-10 makes this mistake, causing it to do much more work than necessary. This loops through several different numbers and writes out each one using the currency format of each culture that uses a comma as a decimal separator.

NOTE

If you run this on Windows, you may find that most of the lines this code displays will contain `?` characters, indicating that the console cannot display most of the currency symbols. In fact, it can—it just needs permission. By default, the Windows console uses an 8-bit code page for backward-compatibility reasons. If you run the command `chcp 65001` from a Command Prompt, it will switch that console window into a UTF-8 code page, enabling it to show any Unicode characters supported by your chosen console font. You might want to configure the console to use a font with comprehensive support for uncommon characters—Consolas or Lucida Console, for example—to take best advantage of that.

Example 10-10. Accidental reevaluation of a deferred query

```
IEnumerable<CultureInfo> commaCultures =
    from culture in CultureInfo.GetCultures(CultureTypes.AllCultures)
    where culture.NumberFormat.NumberDecimalSeparator == ","
    select culture;

object[] numbers = [1, 100, 100.2, 10000.2];
```

```
foreach (object number in numbers)
{
    foreach (CultureInfo culture in commaCultures)
    {
        Console.WriteLine(string.Format(culture, "{0}: {1:c}",
                          culture.Name, number));
    }
}
```

The problem with this code is that even though the `commaCultures` variable is initialized outside of the number loop, we iterate through it for each number. And because LINQ to Objects uses deferred evaluation, that means that the actual work of running the query is redone every time around the outer loop. So, instead of evaluating that `where` clause once for each culture (869 times on my system), it ends up running four times for each culture (3,476 times) because the whole query is evaluated once for each of the four items in the `numbers` array. It's not a disaster—the code still works correctly. But if you do this in a program that runs on a heavily loaded server, it will harm your throughput.

If you know you will need to iterate through the results of a query multiple times, consider using either the `ToList` or `ToArray` extension methods provided by LINQ to Objects. These immediately evaluate the whole query once, producing an `IList<T>` or a `T[]` array, respectively (so you shouldn't use these methods on infinite sequences, obviously). You can then iterate through that as many times as you like without incurring any further costs (beyond the minimal cost inherent in reading array or list elements). But in cases where you iterate through a query only once, it is usually better not to use these methods, as they'll consume more memory than necessary.

LINQ, Generics, and IQueryable<T>

LINQ providers use generic types. LINQ to Objects uses `IEnumerable<T>`. Several of the database providers use a type called `IQueryable<T>`. More broadly, the pattern is to have some generic type `Source<T>`, where `Source` represents some source of items, and `T` is the type of an individ-

ual item. A source type with LINQ support makes operator methods available on `Source<T>` for any `T`, and those operators also typically return `Source<TResult>`, where `TResult` may or may not be different than `T`.

`IQueryable<T>` is interesting because it is designed to be used by multiple providers. This interface, its base `IQueryable`, and the related `IQueryProvider` are shown in Example 10-11.

Example 10-11. `IQueryable` and `IQueryable<T>`

```
public interface IQueryable : IEnumerable
{
    Type ElementType { get; }
    Expression Expression { get; }
    IQueryProvider Provider { get; }
}

public interface IQueryable<out T> : IEnumerable<T>, IQueryable
{
}

public interface IQueryProvider
{
    IQueryable CreateQuery(Expression expression);
    IQueryable<TElement> CreateQuery<TElement>(Expression expression);
    object? Execute(Expression expression);
    TResult Execute<TResult>(Expression expression);
}
```

The most obvious feature of `IQueryable<T>` is that it adds no members to its bases. That's because it's designed to be used entirely via extension methods. The `System.Linq` namespace defines all of the standard LINQ operators for `IQueryable<T>` as extension methods provided by the `Queryable` class. However, all of these simply defer to the `Provider` property defined by the `IQueryable` base. So, unlike LINQ to Objects, where the extension methods on `IEnumerable<T>` define the behavior, an `IQueryable<T>` implementation is able to decide how to handle queries because it gets to supply the `IQueryProvider` that does the real work.

However, all `IQueryable<T>`-based LINQ providers have one thing in common: they interpret the lambdas as expression objects, not delegates. Example 10-12 shows the declaration of the `Where` extension methods defined for `IEnumerable<T>` and `IQueryable<T>`. Compare the `predicate` parameters.

Example 10-12. `Enumerable` versus `Queryable`

```
public static class Enumerable
{
    public static IEnumerable<TSource> Where<TSource>(
        this IEnumerable<TSource> source,
        Func<TSource, bool> predicate)
    ...
}

public static class Queryable
{
    public static IQueryable<TSource> Where<TSource>(
        this IQueryable<TSource> source,
        Expression<Func<TSource, bool>> predicate)
    ...
}
```

The `Where` extension for `IEnumerable<T>` (LINQ to Objects) takes a `Func<TSource, bool>`, and as you saw in Chapter 9, this is a delegate type. But the `Where` extension method for `IQueryable<T>` (used by numerous LINQ providers) takes `Expression<Func<TSource, bool>>`, and as you also saw in Chapter 9, this causes the compiler to build an object model of the expression and pass that as the argument.

A LINQ provider typically uses `IQueryable<T>` if it wants these expression trees. And that's usually because it's going to inspect your query and convert it into something else, such as a SQL query.

There are some other common generic types that crop up in LINQ. Some LINQ features guarantee to produce items in a certain order, and some do not. More subtly, a handful of operators produce items in an order that depends upon the order of their input. This can be reflected in the types

for which the operators are defined and the types they return. LINQ to Objects defines `IOrderedEnumerable<T>` to represent ordered data, and there's a corresponding `IOrderedQueryable<T>` type for `IQueryable<T>`-based providers. (Providers that use their own types tend to do something similar—Parallel LINQ, described in Chapter 16, defines an `OrderedParallelQuery<T>`, for example.) These derive from their unordered counterparts, such as `IEnumerable<T>` and `IQueryable<T>`, so all the usual operators are available, but they make it possible to define operators or other methods that need to take the existing order of their input into account. For example, in "Ordering", I will show a LINQ operator called `ThenBy`, which is available only on sources that are already ordered.

When looking at LINQ to Objects, this ordered/unordered distinction may seem unnecessary, because `IEnumerable<T>` always produces items in some sort of order. But some providers do not necessarily do things in any particular order, perhaps because they parallelize query execution, or because they get a database to execute the query for them, and databases reserve the right to meddle with the order in certain cases if it enables them to work more efficiently.

Standard LINQ Operators

In this section, I will describe the standard operators that LINQ providers can supply. Where applicable, I will also describe the query expression equivalent, although many operators do not have a corresponding query expression form. Some LINQ features are available only through explicit method invocation. This is even true with certain operators that can be used in query expressions, because most operators are overloaded, and query expressions can't use some of the more advanced overloads.

NOTE

LINQ operators are not operators in the usual C# sense—they are not symbols such as `+` or `&&`. LINQ has its own terminology, and for this chapter, an operator is a query capability offered by a LINQ provider. In C#, it looks like a method.

All of these operators have something in common: they have all been designed to support composition. This means that you can combine them in almost any way you like, making it possible to build complex queries out of simple elements. To enable this, operators not only take some type representing a set of items (e.g., an `IEnumerable<T>`) as their input, but most of them also return something representing a set of items. As already mentioned, the item type is not always the same—an operator might take some `IEnumerable<T>` as input, and produce `IEnumerable<TResult>` as output, where `TResult` does not have to be the same as `T`. Even so, you can still chain the things together in any number of ways. Part of the reason this works is that LINQ operators are like mathematical functions in that they do not modify their inputs; rather, they produce a new result that is based on their operands. (Functional programming languages typically have the same characteristic.) This means that not only are you free to plug operators together in arbitrary combinations without fear of side effects, but you are also free to use the same source as the input to multiple queries, because no LINQ query will ever modify its input. Each operator returns a new query based on its input.

Nothing enforces this functional style. The compiler doesn't care what a method representing a LINQ operator does. It is only by convention that operators are functional, in order to support composition, but the built-in LINQ providers all work this way.

Not all providers offer complete support for all operators. The main providers Microsoft supplies—such as LINQ to Objects or the LINQ support in EF Core and Rx—are as comprehensive as they can be, but there are some situations in which certain operators will not make sense.

To demonstrate the operators in action, I need some source data. Many of the examples in the following sections will use the code in Example 10-13.

Example 10-13. Sample input data for LINQ queries

```
public record Course(
    string Title,
    string Category,
    int Number,
```

```
    DateOnly PublicationDate,
    TimeSpan Duration)
{
    public static readonly Course[] Catalog =
    [
        new Course(
            Title: "Elements of Geometry",
            Category: "MAT", Number: 101, Duration: TimeSpan.FromHours(3),
            PublicationDate: new DateOnly(2009, 5, 20)),
        new Course(
            Title: "Squaring the Circle",
            Category: "MAT", Number: 102, Duration: TimeSpan.FromHours(7),
            PublicationDate: new DateOnly(2009, 4, 1)),
        new Course(
            Title: "Recreational Organ Transplantation",
            Category: "BIO", Number: 305, Duration: TimeSpan.FromHours(4),
            PublicationDate: new DateOnly(2002, 7, 19)),
        new Course(
            Title: "Hyperbolic Geometry",
            Category: "MAT", Number: 207, Duration: TimeSpan.FromHours(5),
            PublicationDate: new DateOnly(2007, 10, 5)),
        new Course(
            Title: "Oversimplified Data Structures for Demos",
            Category: "CSE", Number: 104, Duration: TimeSpan.FromHours(2),
            PublicationDate: new DateOnly(2023, 11, 14)),
        new Course(
            Title: "Introduction to Human Anatomy and Physiology",
            Category: "BIO", Number: 201, Duration: TimeSpan.FromHours(12),
            PublicationDate: new DateOnly(2001, 4, 11)),
    ];
}
```

Filtering

One of the simplest operators is `Where`, which filters its input. You provide a predicate, which is a function that takes an individual item and returns a `bool`. `Where` returns an object representing the items from the input for which the predicate is true. (Conceptually, this is very similar to the `FindAll` method available on `List<T>` and array types, but using deferred execution.)

As you've already seen, query expressions represent this with a `where` clause. However, there's an overload of the `Where` operator that provides an additional feature not accessible from a query expression. You can write a filter lambda that takes two arguments: an item from the input and an index representing that item's position in the source. Example 10-14 uses this form to exclude every second item from the input, and it also drops courses shorter than three hours.

Example 10-14. `Where` operator with index

```
IEnumerable<Course> q = Course.Catalog.Where(
    (course, index) => (index % 2 == 0) && course.Duration.TotalHours >= 3);
```

Indexed filtering is meaningful only for ordered data. It always works with LINQ to Objects, because that uses `IEnumerable<T>`, which produces items one after another, but not all LINQ providers process items in sequence. For example, with EF Core, the LINQ queries you write in C# will be handled on the database. Unless a query explicitly requests some particular order, a database is usually free to process items in whatever order it sees fit, possibly in parallel. In some cases, a database may have optimization strategies that enable it to produce the results a query requires using a process that bears little resemblance to the original query. So it might not even be meaningful to talk about, say, the 14th item handled by a `WHERE` clause. Consequently, if you were to write a query similar to Example 10-14 using EF Core, executing the query would cause an exception, complaining that the indexed `Where` operator is not available. If you're wondering why the overload is even present if the provider doesn't support it, it's because EF Core uses `IQueryable<T>`, so all the standard operators are available at compile time; providers that choose to use `IQueryable<T>` can only report the nonavailability of operators at runtime.

NOTE

LINQ providers that implement some or all of the query logic on the server side usually limit what you can do in a query's lambdas. Conversely, LINQ to Objects runs queries in process, so it lets you invoke any method from inside a filter lambda—if you want to call `Console.WriteLine` or read data from a file in your predicate, LINQ to Objects can't stop you. But LINQ providers for databases need to be able to translate your lambdas into something the server can process, so they will reject expressions that use methods with no server-side equivalent.

Even so, you might have expected the exception to emerge when you invoke `Where`, instead of when you try to execute the query (i.e., when you first try to retrieve one or more items). However, providers that convert LINQ queries into some other form, such as a SQL query, typically defer all validation until you execute the query. This is because some operators may be valid only in certain scenarios, meaning that the provider may not know whether any particular operator will work until you've finished building the whole query. It would be inconsistent if errors caused by nonviable queries sometimes emerged while building the query and sometimes when executing it, so even in cases where a provider could determine earlier that a particular operator will fail, it will usually wait until you execute the query to tell you.

The filter lambda you supply to the `Where` operator must take an argument of the item type (the `T` in `IEnumerable<T>`, for example), and it must return a `bool`. You may remember from Chapter 9 that the runtime libraries define a suitable delegate type called `Predicate<T>`, but I also mentioned in that chapter that LINQ avoids this, and we can now see why. The indexed version of the `Where` operator cannot use `Predicate<T>`, because there's an additional argument, so that overload uses `Func<T, int, bool>`. There's nothing stopping the unindexed form of `Where` from using `Predicate<T>`, but LINQ providers tend to use `Func` across the board to ensure that operators with similar meanings have similar-looking signatures. Most providers therefore use `Func<T, bool>` instead, to be consistent with the indexed version. (C# doesn't care which you use—query expressions still work if the provider uses `Predicate<T>`, but none of Microsoft's providers do this.)

WARNING

The C# compiler's nullability analysis doesn't understand what LINQ operators do. Given an `IEnumerable<string?>`, writing `xs.Where(s => s is not null)` removes any null items, but `Where` will still return an `IEnumerable<string?>`. The compiler has no expectations around what `Where` will do, so it doesn't understand that the output is effectively an `IEnumerable<string>`.

LINQ defines another filtering operator: `OfType<T>`. This is useful if your source contains a mixture of different item types—perhaps the source is an `IEnumerable<object>` and you'd like to filter this down to only the items of type `string`. Example 10-15 shows how the `OfType<T>` operator can do this.

Example 10-15. The `OfType<T>` operator

```
public static void ShowAllStrings(IEnumerable<object> src)
{
    foreach (string s in src.OfType<string>())
    {
        Console.WriteLine(s);
    }
}
```

When you use the `OfType<T>` operator with a reference type, it will filter out any `null` values. If you've enabled nullable reference types, `OfType` avoids the problems that `Where(s => s is not null)` encounters: if you call `OfType<string>` on a sequence of type `IEnumerable<string?>`, the resulting type will be `IEnumerable<string>`. But that's not because `OfType` was designed with nullable reference types in mind. On the contrary, it effectively ignores the nullability when you use a reference type as the type argument. It happens to do what we want in this case because it's always looking for a positive match. (It effectively performs the same test as patterns like `o is string`.) The surprising corollary is that `OfType<string?>` will also filter out `null` items, with the slightly peculiar result that it returns an `IEnumerable<string?>` that will never produce a `null`.

Both `Where` and `OfType<T>` will produce empty sequences if none of the objects in the source meets the requirements. This is not considered to be an error—empty sequences are quite normal in LINQ. Many operators can produce them as output, and most operators can cope with them as input.

One last way to filter items is to remove duplicates, for which LINQ defines the `Distinct` operator. Example 10-16 contains a query that extracts the category names from all the courses and then feeds that into the `Distinct` operator to ensure that each unique category name appears just once.

Example 10-16. Removing duplicates with `Distinct`

```
IEnumerable<string> categories =
    Course.Catalog.Select(c => c.Category).Distinct();
```

Select

When writing a query, we may want to extract only certain pieces of data from the source items. The `select` clause at the end of most queries lets us supply a lambda that will be used to produce the final output items, and there are a couple of reasons we might want to make our `select` clause do more than simply pass each item straight through. We might want to pick just one specific piece of information from each item, or we might want to transform it into something else entirely.

You've seen several `select` clauses already, and I showed in Example 10-3 that the compiler turns them into a call to `Select`. However, as with many LINQ operators, the version accessible through a query expression is not the only option. There's one other overload, which provides not just the input item from which to generate the output item but also the index of that item. Example 10-17 uses this to generate a numbered list of course titles.

Example 10-17. `Select` operator with index

```
IEnumerable<string> nonIntro = Course.Catalog.Select((course, index) =>
      $"Course {index}: {course.Title}");
```

Be aware that the zero-based index passed into the lambda will be based on what comes into the `Select` operator and will not necessarily represent the item's original position in the underlying data source. This might not produce the results you were hoping for in code such as Example 10-18.

Example 10-18. Indexed `Select` downstream of `Where` operator

```
IEnumerable<string> nonIntro = Course.Catalog
    .Where(c => c.Number >= 200)
    .Select((course, index) => $"Course {index}: {course.Title}");
```

This code will select the courses found at indexes 2, 3, and 5, respectively, in the `Course.Catalog` array, because those are the courses whose `Number` property satisfies the `Where` expression. However, this query will number the three courses as 0, 1, and 2, because the `Select` operator sees only the items the `Where` clause let through, so the `Select` clause never had access to the original source. As far as it is concerned, there are only three items. If you wanted the indexes relative to the original collection, you'd need to extract those upstream of the `Where` clause, as Example 10-19 shows.

Example 10-19. Indexed `Select` upstream of `Where` operator

```
IEnumerable<string> nonIntro = Course.Catalog
    .Select((course, index) => new { course, index })
    .Where(vars => vars.course.Number >= 200)
    .Select(vars => $"Course {vars.index}: {vars.course.Title}");
```

You may be wondering why I've used an anonymous type here and not a tuple. I could replace `new { course, index }` with just `(course, index)`, and the code would work equally well. (It might even be more ef-

ficient, because tuples are value types, but anonymous types are reference types. Tuples would create less work for the GC here.) However, in general, tuples will not always work in LINQ. The lightweight tuple syntax was introduced in C# 7.0, so they weren't around when expression trees were added back in C# 3.0. The expression object model has not been updated to support this language feature, so if you try to use a tuple with an `IQueryable<T>`-based LINQ provider, you will get compiler error CS8143, telling you that `An expression tree may not contain a tuple literal`. So I tend to use anonymous types in this chapter because they work with query-based providers. But if you're using a purely local LINQ provider (e.g., Rx or LINQ to Objects), feel free to use tuples.

The indexed `Select` operator is similar to the indexed `Where` operator. So, as you would probably expect, not all LINQ providers support it in all scenarios.

Data shaping and anonymous types

If you are using a LINQ provider to access a database, the `Select` operator can offer an opportunity to reduce the quantity of data you fetch, which could reduce the load on your servers. When you use a data access technology such as EF Core to execute a query that returns a set of objects representing persistent entities, there's a trade-off between doing too much work up front and having to do lots of extra deferred work. Should those frameworks fully populate all of the object properties that correspond to columns in various database tables? Should they also load related objects? In general, it's more efficient not to fetch data you're not going to use, and data that is not fetched up front can always be loaded later on demand. However, if you try to be too frugal in your initial request, you may ultimately end up making a lot of extra requests to fill in the gaps, which could outweigh any benefit from avoiding unnecessary work.

When it comes to related entities, EF Core allows you to configure which related entities should be prefetched and which should be loaded on demand, but for any particular entity that gets fetched, all properties relating to columns are typically fully populated. This means queries that request whole entities end up fetching all the columns for any row that they touch.

If you needed to use only one or two columns, fetching them all is relatively expensive. Example 10-20 uses this somewhat inefficient approach. It shows a fairly typical EF Core query.

Example 10-20. Fetching more data than is needed

```
IQueryable<Product> pq = from product in dbCtx.Product
                         where product.ListPrice > 3000
                         select product;
foreach (var prod in pq)
{
    Console.WriteLine($"{prod.Name} ({prod.Size}): {prod.ListPrice}");
}
```

This LINQ provider translates the `where` clause into an efficient SQL equivalent. However, the SQL `SELECT` clause retrieves all the columns from the table. Compare that with Example 10-21. This modifies only one part of the query: the LINQ `select` clause now returns an instance of an anonymous type that contains only those properties we require. (The loop that follows the query can remain the same. It uses `var` for its iteration variable, which will work fine with the anonymous type, which provides the three properties that loop requires.)

Example 10-21. A `select` clause with an anonymous type

```
var pq = from product in dbCtx.Product
         where product.ListPrice > 3000
         select new { product.Name, product.ListPrice, product.Size };
```

The code produces exactly the same results, but it generates a much more compact SQL query that requests only the `Name`, `ListPrice`, and `Size` columns. If you're using a table with many columns, this will produce a significantly smaller response because it's no longer dominated by data we don't need. This reduces the load on the network connection to the database server and also results in faster processing because the data will take less time to arrive. This technique is called *data shaping*.

This approach will not always be an improvement. For one thing, it means you are working directly with data in the database instead of using entity objects. This might mean working at a lower level of abstraction than would be possible if you use the entity types, which might increase development costs. Also, in some environments, database administrators do not allow ad hoc queries, forcing you to use stored procedures, in which case you won't have the flexibility to use this technique.

Projecting the results of a query into an anonymous type is not limited to database queries, by the way. You are free to do this with any LINQ provider, such as LINQ to Objects. It can sometimes be a useful way to get structured information out of a query without needing to define a class specially. (As I mentioned in Chapter 3, anonymous types can be used outside of LINQ, but this is one of the main scenarios for which they were designed. Grouping by composite keys is another, as I'll describe in "Grouping".)

Projection and mapping

The `Select` operator is sometimes referred to as *projection*, and it is the same operation that many languages call *map*, which provides a slightly different way to think about the `Select` operator. So far, I've presented `Select` as a way to choose what comes out of a query, but you can also look at it as a way to apply a transformation to every item in the source. Example 10-22 uses `Select` to produce modified versions of a list of numbers. It variously doubles the numbers, squares them, and turns them into strings.

Example 10-22. Using `Select` to transform numbers

```
int[] numbers = [0, 1, 2, 3, 4, 5];

IEnumerable<int> doubled = numbers.Select(x => 2 * x);
IEnumerable<int> squared = numbers.Select(x => x * x);
IEnumerable<string> numberText = numbers.Select(x => x.ToString());
```

SelectMany

The `SelectMany` LINQ operator is used in query expressions that have multiple `from` clauses. It's called `SelectMany` because, instead of selecting a single output item for each input item, you provide it with a lambda that produces a whole collection for each input item. The resulting query produces all of the objects from all of these collections, as though all of the collections your lambda returns were merged into one. (This won't remove duplicates. Sequences can contain duplicates. If you want to remove them, you can use the `Distinct` operator shown earlier.) There are a couple of ways of thinking about this operator. One is that it provides a means of flattening two levels of hierarchy—a collection of collections—into a single level. Another way to look at it is as a Cartesian product—that is, a way to produce every possible combination from some input sets.

Example 10-23 shows how to use this operator in a query expression. This code highlights the Cartesian-product-like behavior. It shows every combination of the letters A, B, and C with a single digit from 1 to 5—that is, A1, B1, C1, A2, B2, C2, etc. (If you're wondering about the apparent incompatibility of the two input sequences, the `select` clause of this query relies on the fact that if you use the `+` operator to add a `string` and some other type, C# generates code that calls `ToString` on the nonstring operand for you.)

Example 10-23. Using `SelectMany` from a query expression

```
int[] numbers = [1, 2, 3, 4, 5];
string[] letters = ["A", "B", "C"];

IEnumerable<string> combined = from number in numbers
                               from letter in letters
                               select letter + number;
foreach (string s in combined)
{
    Console.WriteLine(s);
}
```

Example 10-24 shows how to invoke the operator directly. This is equivalent to the query expression in Example 10-23.

Example 10-24. `SelectMany` operator

```
IEnumerable<string> combined = numbers.SelectMany(
    number => letters,
    (number, letter) => letter + number);
```

Example 10-23 uses two fixed collections—the second `from` clause returns the same `letters` collection every time. However, you can make the expression in the second `from` clause return a value based on the current item from the first `from` clause. You can see in Example 10-24 that the first lambda passed to `SelectMany` (which actually corresponds to the second `from` clause's final expression) receives the current item from the first collection through its `number` argument, so you can use that to choose a different collection for each item from the first collection. I can use this to exploit `SelectMany`'s flattening behavior.

I've copied a jagged array from Example 5-18 in Chapter 5 into Example 10-25, which then processes it with a query containing two `from` clauses. Note that the expression in the second `from` clause is now `row`, the range variable of the first `from` clause.

Example 10-25. Flattening a jagged array

```
int[][] arrays =
[
    [1, 2],
    [1, 2, 3, 4, 5, 6],
    [1, 2, 4],
    [1],
    [1, 2, 3, 4, 5]
];

IEnumerable<int> flattened = from row in arrays
                             from number in row
                             select number;
```

The first `from` clause asks to iterate over each item in the top-level array. Each of these items is also an array, and the second `from` clause asks to iterate over each of these nested arrays. This nested array's type is `int[]`, so the range variable of the second `from` clause, `number`, represents an `int` from that nested array. The `select` clause just returns each of these `int` values.

The resulting sequence provides every number in the arrays in turn. It has flattened the jagged array into a simple linear sequence of numbers. This behavior is conceptually similar to writing a nested pair of loops, one iterating over the outer `int[][]` array, and an inner loop iterating over the contents of each individual `int[]` array.

The compiler uses the same overload of `SelectMany` for Example 10-25 as it does for Example 10-24, but there's an alternative in this case. The final `select` clause is simpler in Example 10-25—it just passes on items from the second collection unmodified, which means the simpler overload shown in Example 10-26 does the job equally well. With this overload, we just provide a single lambda, which chooses the collection that `SelectMany` will expand for each of the items in the input collection.

Example 10-26. `SelectMany` without item projection

```
IEnumerable<int> flattened = arrays.SelectMany(row => row);
```

That's a somewhat terse bit of code, so in case it's not clear quite how that could end up flattening the array, Example 10-27 shows how you might implement `SelectMany` for `IEnumerable<T>` if you had to write it yourself.

Example 10-27. One implementation of `SelectMany`

```
public static IEnumerable<T2> MySelectMany<T, T2>(
    this IEnumerable<T> src, Func<T, IEnumerable<T2>> getInner)
{
    foreach (T itemFromOuterCollection in src)
    {
        IEnumerable<T2> innerCollection = getInner(itemFromOuterCollection);
```

```
            foreach (T2 itemFromInnerCollection in innerCollection)
            {
                yield return itemFromInnerCollection;
            }
        }
    }
```

Why does the compiler not use the simpler option shown in Example 10-26? The C# language specification defines how query expressions are translated into method calls, and it mentions only the overload shown in Example 10-23. Perhaps the reason the specification doesn't mention the simpler overload is to reduce the demands C# makes of types that want to support this double-`from` query form—you'd need to write only one method to enable this syntax for your own types. However, .NET's various LINQ providers are more generous, providing this simpler overload for the benefit of developers who choose to use the operators directly. In fact, some providers define two more overloads: there are versions of both the `SelectMany` forms we've seen so far that also pass an item index to the first lambda. (The usual caveats about indexed operators apply, of course.)

Although Example 10-27 gives a reasonable idea of what LINQ to Objects does in `SelectMany`, it's not the exact implementation. There are optimizations for special cases. Moreover, other providers may use very different strategies. Databases often have built-in support for Cartesian products, so some providers may implement `SelectMany` in terms of that.

Ordering

In general, LINQ queries do not guarantee to produce items in any particular order unless you explicitly define the order you require. You can do this in a query expression with an `orderby` clause. As Example 10-28 shows, you specify the expression that defines how to order the items and a direction—so this will produce a collection of courses ordered by ascending publication date. As it happens, `ascending` is the default, so you can leave off that qualifier without changing the meaning. As you've probably guessed, you can specify `descending` to reverse the order.

Example 10-28. Query expression with `orderby` clause

```
IOrderedEnumerable<Course> q = from course in Course.Catalog
                               orderby course.PublicationDate ascending
                               select course;
```

The compiler transforms the `orderby` clause in Example 10-28 into a call to the `OrderBy` method, and it would use `OrderByDescending` if you had specified a `descending` sort order. With source types that make a distinction between ordered and unordered collections, these operators return the ordered type (for example, `IOrderedEnumerable<T>` for LINQ to Objects, and `IOrderedQueryable<T>` for `IQueryable<T>`-based providers).

WARNING

With LINQ to Objects, these operators have to retrieve every element from their input before they can produce any output elements. An ascending `OrderBy` can determine which item to return first only once it has found the lowest item, and it won't know for certain which is the lowest until it has seen all of them. It still uses deferred evaluation—it won't do anything until you ask it for the first item. But as soon as you do ask it for something, it has to do all the work at once. Some providers will have additional knowledge about the data that can enable more efficient strategies. (For example, a database may be able to use an index to return values in the order required.)

LINQ to Objects' `OrderBy` and `OrderByDescending` operators each have two overloads, only one of which is available from a query expression. If you invoke the methods directly, you can supply an additional parameter of type `IComparer<TKey>`, where `TKey` is the type of the expression by which the items are being sorted. This is likely to be important if you sort based on a `string` property, because there are several different orderings for text, and you may need to choose one based on your application's locale, or you may want to specify a culture-invariant ordering to ensure consistency across all environments.

The expression that determines the order in Example 10-28 is very simple—it just retrieves the `PublicationDate` property from the source item. You can write more complex expressions if you want to. If you're using a provider that translates a LINQ query into something else, there may be limitations. If the query runs on the database, you may be able to refer to other tables—the provider might be able to convert an expression such as `product.ProductCategory.Name` into a suitable join. However, you will not be able to run any old code in that expression, because it must be something that the database can execute. But LINQ to Objects just invokes the expression once for each object, so you really can put in there whatever code you like.

You may want to sort by multiple criteria. You should *not* do this by writing multiple `orderby` clauses. Example 10-29 makes this mistake.

Example 10-29. How not to apply multiple ordering criteria

```
IOrderedEnumerable<Course> q =
    from course in Course.Catalog
    orderby course.PublicationDate ascending
    orderby course.Duration descending // BAD! Could discard previous order
    select course;
```

This code orders the items by publication date and then by duration but does so as two separate and unrelated steps. The second `orderby` clause guarantees only that the results will be in the order specified in that clause and does not guarantee to preserve anything about the order in which the elements originated. If what you actually wanted was for the items to be in order of publication date, and for any items with the same publication date to be ordered by descending duration, you would need to write the query in Example 10-30.

Example 10-30. Multiple ordering criteria in a query expression

```
IOrderedEnumerable<Course> q =
    from course in Course.Catalog
    orderby course.PublicationDate ascending, course.Duration descending
    select course;
```

LINQ defines separate operators for this multilevel ordering: `ThenBy` and `ThenByDescending`. Example 10-31 shows how to achieve the same effect as the query expression in Example 10-30 by invoking the LINQ operators directly. For LINQ providers whose types make a distinction between ordered and unordered collections, the `ThenBy` and `ThenByDescending` operators will be available only on the ordered form, such as `IOrderedQueryable<T>` or `IOrderedEnumerable<T>`. If you were to try to invoke `ThenBy` directly on `Course.Catalog`, the compiler would report an error.

Example 10-31. Multiple ordering criteria with LINQ operators

```
IOrderedEnumerable<Course> q = Course.Catalog
    .OrderBy(course => course.PublicationDate)
    .ThenByDescending(course => course.Duration);
```

.NET 7.0 added two new ordering operators: `Order` and `OrderDescending`, which can be convenient if you have a collection of items that are inherently comparable. For example, if you had an `IEnumerable<int>`, `OrderBy` looks slightly clunky because it requires a lambda even though you want to sort by the `int` values themselves, and not by some property of these values. You used to have to write `numbers.OrderBy (n => n)`, but now, you can write `numbers.Order()` (or `numbers.OrderDescending()`).

You will find that some LINQ operators preserve some aspects of ordering even if you do not ask them to. For example, LINQ to Objects will typically produce items in the same order in which they appeared in the input unless you write a query that causes it to change the order. But this is simply an artifact of how LINQ to Objects works, and you should not rely on it in general. In fact, even when you are using that particular LINQ provider, you should check with the documentation to see whether the order you're getting is guaranteed or is just an accident of implementation. In most cases, if you care about the order, you should write a query that makes that explicit.

Containment Tests

LINQ defines various standard operators for discovering things about what the collection contains. Some providers may be able to implement these operators without needing to inspect every item. (For example, a database-based provider might use a `WHERE` clause, and the database could be able to use an index to evaluate that without needing to look at every element.) However, there are no restrictions—you can use these operators however you like, and it's up to the provider to discover whether it can exploit a shortcut.

NOTE

Unlike most LINQ operators, in the majority of providers these return neither a collection nor an item from their input. They generally just return `true` or `false`, or in some cases, a count. Rx is a notable exception: its implementations of these operators wrap the `bool` or `int` in a single-element `IObservable<T>` that produces the result. It does this to preserve the reactive nature of processing in Rx.

`Contains`

Takes a single item, and returns `true` if the source contains the specified item and `false` if it does not.

`Any`

Takes an optional predicate, and returns `true` if the predicate is true for at least one item in the source. If you do not provide the predicate, this returns `true` if the source contains at least one item.

`Count` and `LongCount`

Take an optional predicate, and return the number of elements in the source for which the predicate is true. If you do not provide the predicate, these return the number of elements in the source. `Count` returns an `int`, so you would use `LongCount` only when dealing with very large collections. (`LongCount` is likely to be

overkill for most LINQ to Objects applications, but it could matter when the collection lives in a database.)

`All`

Takes a predicate, and it returns `true` if and only if the source contains no items that do not match the predicate. (I've used this slightly awkward phrasing for a reason: this returns `true` for an empty sequence. This is consistent with the mathematical logical operator that `All` represents: the *universal quantifier*, usually written as an upside-down A (∀) and pronounced "for all." Mathematicians long ago agreed on the convention that applying the universal quantifier to an empty set yields the value `true`.)

You should be wary of code such as `if (q.Count() > 0)`. Calculating the exact count may require the entire source query (`q` in this case) to be evaluated, and in any case, it is likely to require more work than simply answering the question, *Is this empty?* If `q` refers to a LINQ query, writing `if (q.Any())` is likely to be more efficient. That said, outside of LINQ, this is not the case for list-like collections. If `q` were an `IList<T>`, for example, retrieving an element count would be cheap and may actually be more efficient than the `Any` operator.

There are some situations in which you might want to use a count only if one can be calculated efficiently. (For example, a user interface might want to show the total number of items available if this is easy to determine, but could easily choose not to show it for cases where that would be too expensive.) For these scenarios, you can use the `TryGetNonEnumeratedCount` method. This will return `true` if the count can be determined without having to iterate through the whole collection, and `false` if not. When it returns `true`, it passes the count back through its single argument of type `out int`.

ASYNCHRONOUS IMMEDIATE EVALUATION

Although most LINQ operators defer execution, as you've now seen there are some exceptions. With most LINQ providers, the `Contains`, `Any`, and `All` operators do not produce a wrapped result. (E.g., in LINQ to Objects, these return a `bool`, not an `IEnumerable<bool>`.) This sometimes means that these operators need to do some slow work. For example, EF Core's LINQ provider will need to send a query to the database and wait for the response before being able to return the `bool` result. The same goes for `ToArray` and `ToList`, which produce fully populated collections, instead of an `IEnumerable<T>` or `IQueryable<T>` that have the potential to produce results in the future.

As Chapter 16 describes, it is common for slow operations like these to implement the Task-based Asynchronous Pattern (TAP), enabling us to use the `await` keyword described in Chapter 17. Some LINQ providers therefore choose to offer asynchronous versions of these operators. For example, EF Core offers `SingleAsync`, `ContainsAsync`, `AnyAsync`, `AllAsync`, `ToArrayAsync`, and `ToListAsync`, and equivalents for the other operators we'll see that perform immediate evaluation.

Specific Items and Subranges

It can be useful to write a query that produces just a single item. Perhaps you're looking for the first object in a list that meets certain criteria, or maybe you want to fetch information in a database identified by a particular key. LINQ defines several operators that can do this and some related ones for working with a subrange of the items a query might return.

Use the `Single` operator when you have a query that you believe should produce exactly one result. Example 10-32 shows just such a query—it looks up a course by its category and number, and in my sample data, this uniquely identifies a course.

Example 10-32. Applying the `Single` operator to a query

```
IEnumerable<Course> q = from course in Course.Catalog
                        where course.Category == "MAT" && course.Number == 101
```

```
                        select course;

Course geometry = q.Single();
```

Because LINQ queries are built by chaining operators together, we can take the query built by the query expression and add on another operator—the `Single` operator, in this case. While most operators would return an object representing another query (an `IEnumerable<T>` here, since we're using LINQ to Objects), `Single` is different. Like `ToArray` and `ToList`, the `Single` operator evaluates the query immediately, but it then returns the one and only object that the query produced. If the query fails to produce exactly one object—perhaps it produces no items, or two—this will throw an `InvalidOperationException`. (Since this is another of the operators that produces a result immediately, some providers offer `SingleAsync` as described in the sidebar "Asynchronous Immediate Evaluation".)

There's an overload of the `Single` operator that takes a predicate. As Example 10-33 shows, this allows us to express the same logic as the whole of Example 10-32 more compactly. (As with the `Where` operator, all the predicate-based operators in this section use `Func<T, bool>`, not `Predicate<T>`.)

Example 10-33. The `Single` operator with predicate

```
Course geometry = Course.Catalog.Single(
    course => course.Category == "MAT" && course.Number == 101);
```

The `Single` operator is unforgiving: if your query does not return exactly one item, it will throw an exception. There's a slightly more flexible variant called `SingleOrDefault`, which allows a query to return either one item or no items. If the query returns nothing, this method returns the default value for the item type (i.e., `null` if it's a reference type, `0` if it's a numeric type, etc.). Multiple matches still cause an exception. As with `Single`, there are two overloads: one with no arguments for use on a source that you believe contains no more than one object, and one that takes a predicate lambda.

LINQ defines two related operators, `First` and `FirstOrDefault`, each of which offers overloads taking no arguments or a predicate. For sequences containing zero or one matching items, these behave in exactly the same way as `Single` and `SingleOrDefault`: they return the item if there is one; if there isn't, `First` will throw an exception, while `FirstOrDefault` will return `null` or an equivalent value. However, these operators respond differently when there are multiple results—instead of throwing an exception, they just pick the first result and return that, discarding the rest. This might be useful if you want to find the most expensive item in a list—you could order a query by descending price and then pick the first result. Example 10-34 uses a similar technique to pick the longest course from my sample data.

Example 10-34. Using `First` to select the longest course

```
IOrderedEnumerable<Course> q = from course in Course.Catalog
                               orderby course.Duration descending
                               select course;
Course longest = q.First();
```

If you have a query that doesn't guarantee any particular order for its results, these operators will pick one item arbitrarily.

TIP

Do not use `First` or `FirstOrDefault` unless you expect there to be multiple matches and you want to process only one of them. Some developers use these when they expect only a single match. The operators will work, of course, but the `Single` and `SingleOrDefault` operators more accurately express your expectations. They will let you know when your expectations were misplaced, throwing an exception when there are multiple matches. If your code embodies incorrect assumptions, it's usually best to know about it instead of plowing on regardless.

The existence of `First` and `FirstOrDefault` raises an obvious question: Can I pick the last item? The answer is yes; there are also `Last` and `LastOrDefault` operators, and again, each offers two overloads—one taking no arguments and one taking a predicate.

The `SingleOrDefault`, `FirstOrDefault`, and `LastOrDefault` operators each offer an overload enabling you to supply a value to return as the default, instead of the usual zero-like value. Example 10-35 shows how to use this `SingleOrDefault` overload to get a value of −1 when the list is empty, making it possible to distinguish between an empty list and a list containing a single zero value. (Of course, if all possible values for `int` are valid in your application, this doesn't help you, and you'd need to detect an empty collection in some other way. But in cases where you can designate some special value to represent *not here* [e.g., −1 in this case], these overloads are helpful.)

Example 10-35. `SingleOrDefault` with explicit default value

```
int valueOrNegative = numbers.SingleOrDefault(-1);
```

The next obvious question is: What if I want a particular element that's neither the first nor the last? Your wish is, in this particular instance, LINQ's command, because it offers `ElementAt` and `ElementAtOrDefault` operators, both of which take just an index. This provides a way to access elements of any `IEnumerable<T>` by index. You can specify the index as an `int`. Alternatively, some providers define overloads taking an `Index`, which, as you may recall from "Addressing Elements with Index and Range Syntax", enables the use of end-relative positions. For example, `^2` denotes the second-from-last element.

You need to be careful with `ElementAt` and `ElementAtOrDefault` because they can be surprisingly expensive. If you ask for the 10,000th element, these operators may need to request and discard the first 9,999 elements to get there. If you specify an end-relative position by writing, say, `source.ElementAt(^500)`, the operator may need to read every single element to find out which is the last, and with that particular example, it may also have to hang on to the last 500 elements it has seen because until it gets to the end, it doesn't know which element will be the one it ultimately has to return.

As it happens, LINQ to Objects detects when the source object implements `IList<T>`, in which case it uses the indexer to retrieve the element di-

rectly instead of going the slow way around. But not all `IEnumerable<T>` implementations support random access, so these operators can be very slow. In particular, even if your source implements `IList<T>`, once you've applied one or more LINQ operators to it, the output of those operators will typically not support indexing. So it would be particularly disastrous to use `ElementAt` in a loop of the kind shown in Example 10-36.

Example 10-36. How not to use `ElementAt`

```
IEnumerable<Course> mathsCourses =
    Course.Catalog.Where(c => c.Category == "MAT");
for (int i = 0; i < mathsCourses.Count(); ++i)
{
    // Never do this!
    Course c = mathsCourses.ElementAt(i);
    Console.WriteLine(c.Title);
}
```

Even though `Course.Catalog` is an array, I've filtered its contents with the `Where` operator, which returns a query of type `IEnumerable<Course>` that does not implement `IList<Course>`. The first iteration won't be too bad—I'll be passing `ElementAt` an index of `0`, so it just returns the first match, and with my sample data, the very first item `Where` inspects will match. But the second time around the loop, we're calling `ElementAt` again. The query that `mathsCourses` refers to does not keep track of where we got to in the previous loop—it's an `IEnumerable<T>`, not an `IEnumerator<T>`—so this will start again. `ElementAt` will ask that query for the first item, which it will promptly discard, and then it will ask for the next item, and that becomes the return value. So the `Where` query has now been executed twice—the first time, `ElementAt` asked it for only one item, and then the second time it asked it for two, so it has processed the first course twice now. The third time around the loop (which happens to be the final time), we do it all again, but this time, `ElementAt` will discard the first two matches and will return the third, so now it has looked at the first course three times, the second one twice, and the third and fourth courses once. (The third course in my sample data is not in the `MAT` category, so the `Where` query will skip over this when asked for the third item.) So, to retrieve three

items, I've evaluated the `Where` query three times, causing it to evaluate my filter lambda seven times.

In fact, it's worse than that, because the `for` loop will also invoke that `Count` method each time, and with a nonindexable source such as the one returned by `Where`, `Count` has to evaluate the entire sequence—the only way the LINQ to Objects `Where` operator can tell you how many items match is to look at all of them. So this code fully evaluates the query returned by `Where` three times in addition to the three partial evaluations performed by `ElementAt`. We get away with it here because the collection is small, but if I had an array with 1,000 elements, all of which turned out to match the filter, we'd be fully evaluating the `Where` query 1,000 times and performing partial evaluations another 1,000 times. Each full evaluation calls the filter predicate 1,000 times, and the partial evaluations here will do so on average 500 times, so the code would end up executing the filter 1,500,000 times. Iterating through the `Where` query with the `foreach` loop would evaluate the query just once, executing the filter expression 1,000 times, and would produce the same results.

So be careful with both `Count` and `ElementAt`. If you use them in a loop that iterates over the collection on which you invoke them, the resulting code will have $O(n^2)$ complexity (i.e., the cost of running the code rises proportionally to the number of items squared).

All of the operators I've just described return a single item from the source. There are four more operators that also get selective about which items to use but can return multiple items: `Skip`, `Take`, `SkipLast`, and `TakeLast`. Each of these takes a single `int` argument. As the name suggests, `Skip` discards the specified number of elements from the beginning of the sequence and then returns everything else from its source. `Take` returns the specified number of elements from the start of the sequence and then discards the rest (so it is similar to `TOP` in SQL). `SkipLast` and `TakeLast` do the same except they work at the end, e.g., you could use `TakeLast` to get the final five items from the source, or `SkipLast` to omit the final five items.

Some providers (including LINQ to Objects) supply an overload of `Take` that accepts a `Range`, enabling the use of the range syntax described in

"Addressing Elements with Index and Range Syntax". For example, `source.Take(10..^10)` is equivalent to `source.Skip(10).SkipLast(10)`, skipping the first 10 and also the last 10 items. Since the range syntax lets you use either start- or end-relative indexes for both the start and end of the range, we can express other combinations with this overload of `Take`. For example, `source.Take(10..20)` has the same effect as `source.Skip(10).Take(10)`; `source.Take(^10..^2)` is equivalent to `source.TakeLast(10).SkipLast(2)`.

There are also predicate-driven versions, `SkipWhile` and `TakeWhile`. `SkipWhile` will discard items from the sequence until it finds one that does not match the predicate, at which point it will return that and every item that follows for the rest of the sequence (whether or not the remaining items match the predicate). Conversely, `TakeWhile` returns items until it encounters the first item that does not match the predicate, at which point it discards that and the remainder of the sequence.

Although `Skip`, `Take`, `SkipLast`, `TakeLast`, `SkipWhile`, and `TakeWhile` are all clearly order-sensitive, they are not restricted to just the ordered types, such as `IOrderedEnumerable<T>`. They are also defined for `IEnumerable<T>`, which is reasonable, because even though there may be no particular order guaranteed, an `IEnumerable<T>` always produces elements in some order. (The only way you can extract items from an `IEnumerable<T>` is one after another, so there will always be an order, even if it's arbitrary. It might not be the same every time you enumerate the items, but for any single evaluation, the items must come out in some order.) Moreover, `IOrderedEnumerable<T>` is not widely implemented outside of LINQ, so it's quite common to have non-LINQ-aware objects that produce items in a known order but that implement only `IEnumerable<T>`. These operators are useful in these scenarios, so the restriction is relaxed. Slightly more surprisingly, `IQueryable<T>` also supports these operations, but that's consistent with the fact that many databases support `TOP` (roughly equivalent to `Take`) even on unordered queries. As always, individual providers may choose not to support individual operations, so in scenarios where there's no reasonable interpretation of these operators, they will just throw an exception.

Whole-Sequence, Order-Preserving Operations

LINQ defines certain operators whose output includes every item from the source, and that preserve or reverse the order. Not all collections necessarily have an order, so these operators will not always be supported. However, LINQ to Objects supports all of them:

`Concat`

Combines two sequences, producing all of the elements from the first sequence (in whatever order that sequence returns them), followed by all of the elements from the second sequence (again, preserving the order).

`DefaultIfEmpty`

Returns all of the elements from the source. However, if the source is empty, it returns a single element. If you don't pass the value to return as the default, this uses `default(TElement)`.

`Prepend` and `Append`

Returns all the same elements as the source sequence but with one additional element at start or end, respectively.

`Reverse`

Reverses the order of the elements.

`SequenceEqual`

Compares two sequences. Returns `true` if they are the same length and contain the same values in the same order.

`Zip`

Combines two sequences, pairing elements. The first item it returns will be based on both the first item from the first sequence and the first item from the second sequence. The second item in the zipped sequence will be based on the second items from each of the sequences, and so on. (The name `Zip` is meant to bring to mind how a zipper in an article of clothing brings two things together in per-

fect alignment. It's not an exact analogy. When a zipper brings together the two parts, the teeth from the two halves interlock in an alternating fashion. But the `Zip` operator does not interleave its inputs like a physical zipper's teeth. It brings items from the two sources together in pairs.) Some providers also define an overload for combining three lists.

Aggregation

The `Sum` and `Average` operators add together the values of all the source items. `Sum` returns the total, and `Average` returns the total divided by the number of items. LINQ providers that support these typically make them available for collections of items of these numeric types: `decimal`, `double`, `float`, `int`, and `long`. There are also overloads that work with any item type in conjunction with a lambda that takes an item and returns one of those numeric types. That allows us to write code such as Example 10-37, which works with a collection of `Course` objects and calculates the average of a particular value extracted from the object: the course length in hours.

Example 10-37. `Average` operator with projection

```
Console.WriteLine("Average course length in hours: {0}",
    Course.Catalog.Average(course => course.Duration.TotalHours));
```

LINQ also defines `Min` and `Max` operators. You can apply these to any type of sequence, although it is not guaranteed to succeed—the particular provider you're using may report an error if it doesn't know how to compare the types you've used. For example, LINQ to Objects requires the objects in the sequence to implement `IComparable`.

`Min` and `Max` both have overloads that accept a lambda that gets the value to use from the source item. Example 10-38 uses this to find the date on which the most recent course was published.

Example 10-38. `Max` with projection

```
DateOnly m = mathsCourses.Max(c => c.PublicationDate);
```

Notice that this does not return the course with the most recent publication date; it returns that course's publication date. If you want to select the object for which a particular property has the maximum value, you can use `MaxBy`. Example 10-39 will find the course with the highest `PublicationDate`, but unlike Example 10-38, it returns the relevant course, instead of the date. (As you might expect, there's also a `MinBy`.)

Example 10-39. `MaxBy` with projection for criteria but not for result

```
Course? mostRecentlyPublished = mathsCourses.MaxBy(c => c.PublicationDate);
```

You may have spotted the `?` in that example, indicating that `MaxBy` might return a `null` result. This happens with both `Max` and `MaxBy` in cases where the input collection is empty and the output type is either a reference type or a nullable form of one of the supported numeric types (e.g., `int?` or `double?`). When the output is a non-nullable struct (e.g., `DateOnly`, as with Example 10-38), these operators cannot return `null` and will throw an `InvalidOperationException` instead. If you are working with a reference type and you want an exception for an empty input like you would get if the output were a value type, the only way to do that is to check for a `null` result yourself and throw an exception. Example 10-40 shows one way to do this.

Example 10-40. `MaxBy` with projection for criteria but not for result, with error on empty input

```
Course mostRecentlyPublished = mathsCourses.MaxBy(c => c.PublicationDate)
    ?? throw new InvalidOperationException("Collection must not be empty");
```

LINQ to Objects defines specialized overloads of `Min` and `Max` for sequences that return the same numeric types that `Sum` and `Average` deal with (i.e., `decimal`, `double`, `float`, `int`, and `long` and their nullable forms). It also defines similar specializations for the form that takes a

lambda. These overloads exist to improve performance by avoiding boxing. The general-purpose form relies on `IComparable`, and getting an interface type reference to a value always involves boxing that value. For large collections, boxing every single value would put considerable extra pressure on the GC.

LINQ defines an operator called `Aggregate`, which generalizes the pattern that `Min`, `Max`, `Sum`, and `Average` all use, which is to produce a single result with a process that involves taking every source item into consideration. It's possible to implement all four of these operators (and their `...By` counterparts) in terms of `Aggregate`. Example 10-41 uses the `Sum` operator to calculate the total duration of all courses, and then shows how to use the `Aggregate` operator to perform the exact same calculation.

Example 10-41. `Sum` and equivalent with `Aggregate`

```
double t1 = Course.Catalog.Sum(course => course.Duration.TotalHours);
double t2 = Course.Catalog.Aggregate(
    0.0, (hours, course) => hours + course.Duration.TotalHours);
```

Aggregation works by building up a value that represents what we know about all the items inspected so far, referred to as the *accumulator*. The type we use depends on the knowledge we want to accumulate. Here, I'm just adding all the numbers together, so I've used a `double` (because the `TimeSpan` type's `TotalHours` property is also a `double`).

Initially we have no knowledge, because we haven't looked at any items yet. We need to provide an accumulator value to represent this starting point, so the `Aggregate` operator's first argument is the *seed*, an initial value for the accumulator. In Example 10-41, the accumulator is just a running total, so the seed is `0.0`.

The second argument is a lambda that describes how to update the accumulator to incorporate information for a single item. Since my goal here is simply to calculate the total time, I just add the duration of the current course to the running total.

Once `Aggregate` has looked at every item, this particular overload returns the accumulator directly. It will be the total number of hours across all courses in this case. We can implement `Max` if we use a different accumulation strategy. Instead of maintaining a running total, the value representing everything we know so far about the data is simply the highest value seen yet. Example 10-42 shows the rough equivalent of Example 10-38. (It's not exactly the same, because Example 10-42 makes no attempt to detect an empty source. `Max` will throw an exception if this source is empty, but this will just return the date 0/0/0000.)

Example 10-42. Implementing `Max` with `Aggregate`

```
DateOnly m = mathsCourses.Aggregate(
    new DateOnly(),
    (date, c) => date > c.PublicationDate ? date : c.PublicationDate);
```

This illustrates that `Aggregate` does not impose any single meaning for the value that accumulates knowledge—the way you use it depends on what you're doing. Some operations require an accumulator with a bit more structure. Example 10-43 calculates the average course duration with `Aggregate`.

Example 10-43. Implementing `Average` with `Aggregate`

```
double average = Course.Catalog.Aggregate(
    new { TotalHours = 0.0, Count = 0 },
    (totals, course) => new
    {
        TotalHours = totals.TotalHours + course.Duration.TotalHours,
        Count = totals.Count + 1
    },
    totals => totals.Count > 0
        ? totals.TotalHours / totals.Count
        : throw new InvalidOperationException("Sequence was empty"));
```

The average duration requires us to know two things: the total duration and the number of items. So, in this example, my accumulator uses a type that can contain two values, one to hold the total and one to hold the item count. I've used an anonymous type because as already mentioned, that is

sometimes the only option in LINQ, and I want to show the most general case. However, it's worth mentioning that in this particular case, a tuple might be better. It will work because this is LINQ to Objects, and since lightweight tuples are value types whereas anonymous types are reference types, a tuple would reduce the number of objects being allocated.

NOTE

Example 10-43 relies on the fact that when two separate methods in the same component create instances of two identical anonymous types, the compiler generates a single type that is used for both. The seed produces an instance of an anonymous type consisting of a `double` called `TotalHours` and an `int` called `Count`. The accumulation lambda also returns an instance of an anonymous type with the same member names and types in the same order. The C# compiler deems that these will be the same type, which is important, because `Aggregate` requires the lambda to accept and also return an instance of the accumulator type.

Example 10-43 uses a different overload than the earlier example. It takes an extra lambda, which is used to extract the return value from the accumulator—the accumulator builds up the information I need to produce the result, but the accumulator itself is not the result in this example.

Of course, if all you want to do is calculate the sum, maximum, or average values, you wouldn't use `Aggregate`—you'd use the specialized operators designed to do those jobs. Not only are they simpler, but they're often more efficient. (For example, a LINQ provider for a database might be able to generate a query that uses the database's built-in features to calculate the minimum or maximum value.) I just wanted to show the flexibility, using examples that are easily understood. But now that I've done that, Example 10-44 shows a particularly concise example of `Aggregate` that doesn't correspond to any other built-in operator. This takes a collection of rectangles and returns the bounding box that contains all of those rectangles.

Example 10-44. Aggregating bounding boxes

```
public static Rect GetBounds(IEnumerable<Rect> rects) =>
    rects.Aggregate(Rect.Union);
```

The `Rect` structure in this example is from the `System.Windows` namespace. This is part of WPF, and it's a very simple data structure that just contains four numbers—`X`, `Y`, `Width`, and `Height`—so you can use it in non-WPF applications if you like.[1] Example 10-44 uses the `Rect` type's static `Union` method, which takes two `Rect` arguments and returns a single `Rect` that is the bounding box of the two inputs (i.e., the smallest rectangle that contains both of the input rectangles).

I'm using the simplest overload of `Aggregate` here. It does the same thing as the one I used in Example 10-41, but it doesn't require me to supply a seed—it just uses the first item in the list. Example 10-45 is equivalent to Example 10-44 but makes the steps more explicit. I've provided the first `Rect` in the sequence as an explicit seed value, using `Skip` to aggregate over everything except that first element. I've also written a lambda to invoke the method, instead of passing the method itself. If you're using this sort of lambda that just passes its arguments straight on to an existing method with LINQ to Objects, you can just pass the method name instead, and it will call the target method directly rather than going through your lambda. (You can't do that with expression-based providers, because they require a lambda.)

Using the method directly is more succinct, but it also makes for slightly obscure code, which is why I've spelled it out in Example 10-45.

Example 10-45. More verbose and less obscure bounding box aggregation

```
public static Rect GetBounds(IEnumerable<Rect> rects)
{
    IEnumerable<Rect> theRest = rects.Skip(1);
    return theRest.Aggregate(rects.First(), (r1, r2) => Rect.Union(r1, r2));
}
```

These two examples work the same way. They start with the first rectangle as the seed. For the next item in the list, `Aggregate` will call `Rect.Union`, passing in the seed and the second rectangle. The result—the bounding box of the first two rectangles—becomes the new accumulator value. And that then gets passed to `Union` along with the third rectangle, and so on. Example 10-46 shows what the effect of this `Aggregate` operation would be if performed on a collection of four `Rect` values. (I've represented the four values here as `r1`, `r2`, `r3`, and `r4`. To pass them to `Aggregate`, they'd need to be inside a collection such as an array.)

Example 10-46. The effect of `Aggregate`

```
Rect bounds = Rect.Union(Rect.Union(Rect.Union(r1, r2), r3), r4);
```

`Aggregate` is LINQ's name for an operation some other languages call *reduce*. You also sometimes see it called *fold*. LINQ went with the name `Aggregate` for the same reason it calls its projection operator `Select` instead of *map* (the more common name in functional programming languages): LINQ's terminology is more influenced by SQL than it is by functional programming languages.

Grouping

Sometimes you will want to process all items that have something in common as a group. Example 10-47 uses a query to group courses by category, writing out a title for each category before listing all the courses in that category.

Example 10-47. Grouping query expression

```
IEnumerable<IGrouping<string, Course>> subjectGroups =
    from course in Course.Catalog
    group course by course.Category;

foreach (IGrouping<string, Course> group in subjectGroups)
{
    Console.WriteLine($"Category: {group.Key}");
    Console.WriteLine();
```

```
        foreach (Course course in group)
        {
            Console.WriteLine(course.Title);
        }
        Console.WriteLine();
    }
```

A `group` clause takes an expression that determines group membership—in this case, any courses whose `Category` properties return the same value will be deemed to be in the same group. A `group` clause produces a collection in which each item implements a type representing a group. Since I am using LINQ to Objects, and I am grouping by category string, the type of the `subjectGroup` variable in Example 10-47 will be `IEnumerable<IGrouping<string, Course>>`. This particular example produces three group objects, depicted in Figure 10-1.

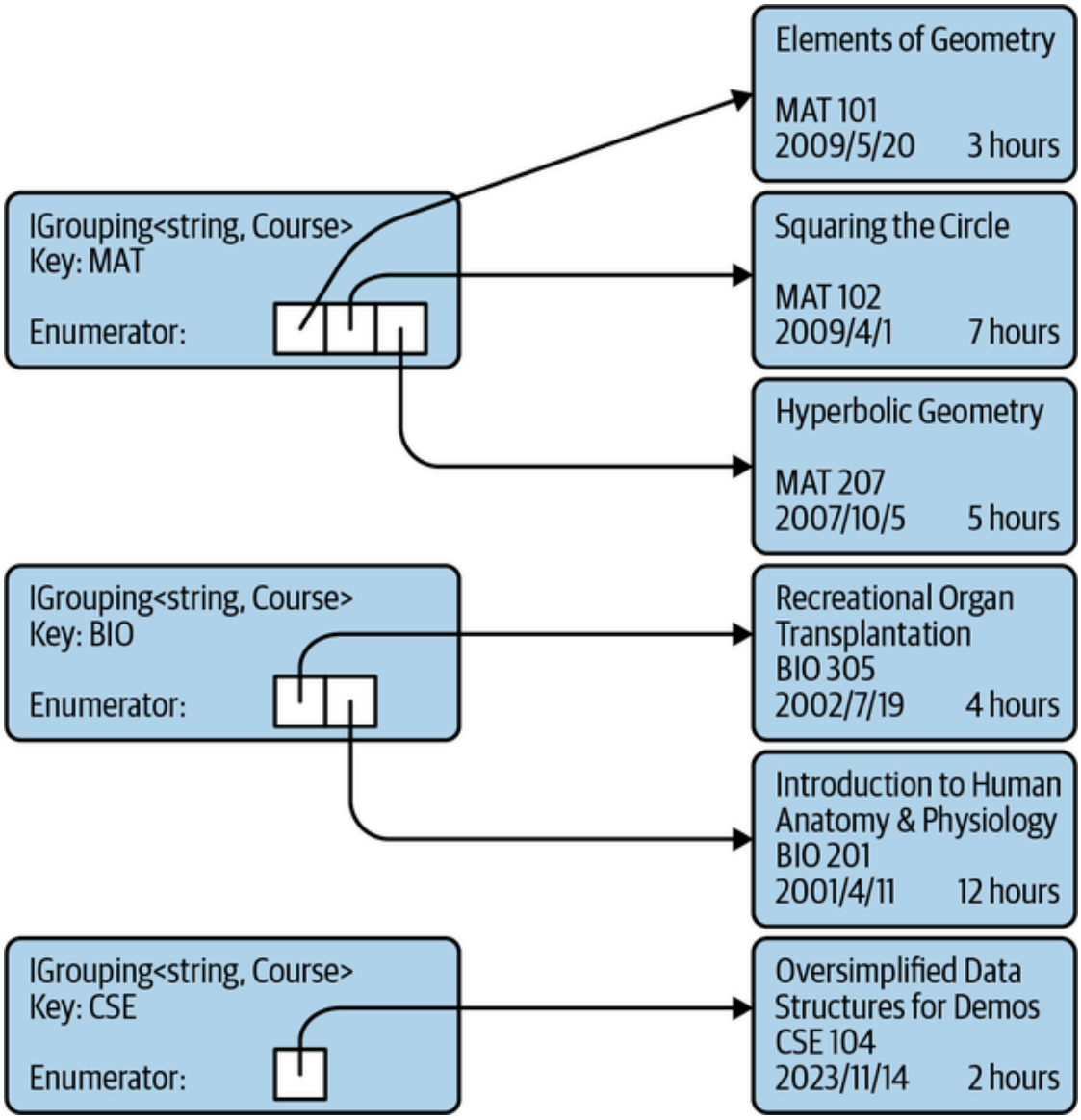

Figure 10-1. Result of evaluating a grouping query

Each of the `IGrouping<string, Course>` items has a `Key` property, and because the query groups items by the course's `Category` property, each key contains a string value from that property. There are three different category names in the sample data in Example 10-13: `MAT`, `BIO`, and `CSE`, so these are the `Key` values for the three groups.

The `IGrouping<TKey, TItem>` interface derives from `IEnumerable<TItem>`, so each group object can be enumerated to find the items it contains. So in Example 10-47, the outer `foreach` loop iterates over the three groups returned by the query, and then the inner `foreach` loop iterates over the `Course` objects in each of the groups.

The query expression turns into the code in Example 10-48.

Example 10-48. Expanding a simple grouping query

```
IEnumerable<IGrouping<string, Course>> subjectGroups =
    Course.Catalog.GroupBy(course => course.Category);
```

Query expressions offer some variations on the theme of grouping. With a slight modification to the original query, we can arrange for the items in each group to be something other than the original `Course` objects. In Example 10-49, I've changed the expression immediately after the `group` keyword from just `course` to `course.Title`.

Example 10-49. Group query with item projection

```
IEnumerable<IGrouping<string, string>> subjectGroups =
    from course in Course.Catalog
    group course.Title by course.Category;
```

This still has the same grouping expression, `course.Category`, so this produces three groups as before, but now it's of type `IGrouping<string, string>`. If you were to iterate over the contents of one of the groups, you'd find each group offers a sequence of strings, containing the course names. As Example 10-50 shows, the compiler expands this query into a different overload of the `GroupBy` operator.

Example 10-50. Expanding a group query with an item projection

```
IEnumerable<IGrouping<string, string>> subjectGroups = Course.Catalog
    .GroupBy(course => course.Category, course => course.Title);
```

Query expressions are required to have either a `select` or a `group` as their final clause. However, if a query contains a `group` clause, that doesn't have to be the last clause. In Example 10-49, I modified how the query represents each item within a group (i.e., the boxes on the right of Figure 10-1), but I'm also free to customize the objects representing each group (the items on the left). By default, I get the `IGrouping<TKey, TItem>` objects (or the equivalent for whichever LINQ provider the query is using), but I can change this. Example 10-51 uses the optional `into`

keyword in its `group` clause. This introduces a new range variable, which iterates over the group objects, which I can go on to use in the rest of the query. I could follow this with other clause types, such as `orderby` or `where`, but in this case, I've chosen to use a `select` clause.

Example 10-51. Group query with group projection

```
IEnumerable<string> subjectGroups =
    from course in Course.Catalog
    group course by course.Category into category
    select $"Category '{category.Key}' contains {category.Count()} courses";
```

The result of this query is an `IEnumerable<string>`, and if you display all the strings it produces, you get this:

```
Category 'MAT' contains 3 courses
Category 'BIO' contains 2 courses
Category 'CSE' contains 1 courses
```

As Example 10-52 shows, this expands into a call to the same `GroupBy` overload that Example 10-48 uses, and then uses the ordinary `Select` operator for the final clause.

Example 10-52. Expanded group query with group projection

```
IEnumerable<string> subjectGroups = Course.Catalog
    .GroupBy(course => course.Category)
    .Select(category =>
        $"Category '{category.Key}' contains {category.Count()} courses");
```

LINQ to Objects defines some more overloads for the `GroupBy` operator that are not accessible from the query syntax. Example 10-53 shows an overload that provides a slightly more direct equivalent to Example 10-51.

Example 10-53. `GroupBy` with key and group projections

```
IEnumerable<string> subjectGroups = Course.Catalog.GroupBy(
    course => course.Category,
    (category, courses) =>
        $"Category '{category}' contains {courses.Count()} courses");
```

This overload takes two lambdas. The first is the expression by which items are grouped. The second is used to produce each group object. Unlike the previous examples, this does not use the `IGrouping<TKey, TItem>` interface. Instead, the final lambda receives the key as one argument and then a collection of the items in the group as the second. This is exactly the same information that `IGrouping<TKey, TItem>` encapsulates, but because this form of the operator can pass these as separate arguments, it removes the need for the operator to create objects to represent the groups.

There's yet another version of this operator shown in Example 10-54. It combines the functionality of all the other flavors.

Example 10-54. `GroupBy` operator with key, item, and group projections

```
IEnumerable<string> subjectGroups = Course.Catalog.GroupBy(
    course => course.Category,
    course => course.Title,
    (category, titles) =>
        $"Category '{category}' contains {titles.Count()} courses: " +
            string.Join(", ", titles));
```

This overload takes three lambdas. The first is the expression by which items are grouped. The second determines how individual items in a group are represented—this time I've chosen to extract the course title. The third lambda is used to produce each group object, and as with Example 10-53, this final lambda is passed the key as one argument, and its other argument gets the group items, as transformed by the second lambda. So, rather than the original `Course` items, this second argument will be an `IEnumerable<string>` containing the course titles, because

that's what the second lambda in this example requested. The result of this `GroupBy` operator is once again a collection of strings, but now it looks like this:

```
Category 'MAT' contains 3 courses: Elements of Geometry, Squaring the Circle, Hy
perbolic Geometry
Category 'BIO' contains 2 courses: Recreational Organ Transplantation, Introduc
tion to Human Anatomy and Physiology
Category 'CSE' contains 1 courses: Oversimplified Data Structures for Demos
```

I've shown four versions of the `GroupBy` operator. All four take a lambda that selects the key to use for grouping, and the simplest overload takes nothing else. The others let you control the representation of individual items in the group, or the representation of each group, or both. There are four more versions of this operator. They offer all the same services as the four I've shown already but also take an `IEqualityComparer<T>`, which lets you customize the logic that decides whether two keys are considered to be the same for grouping purposes.

Sometimes it is useful to group by more than one value. For example, suppose you want to group courses by both category and publication year. You could chain the operators, grouping first by category and then by year within the category (or vice versa). But you might not want this level of nesting—instead of groups of groups, you might want to group courses under each unique combination of `Category` and publication year. You do this by putting both values into the key, and you can do that by using an anonymous type, as Example 10-55 shows.

Example 10-55. Composite group key

```
var bySubjectAndYear =
    from course in Course.Catalog
    group course by new { course.Category, course.PublicationDate.Year };
foreach (var group in bySubjectAndYear)
{
    Console.WriteLine($"{group.Key.Category} ({group.Key.Year})");
    foreach (Course course in group)
    {
        Console.WriteLine(course.Title);
```

```
    }
}
```

This takes advantage of the fact that anonymous types implement `Equals` and `GetHashCode` for us. It works for all forms of the `GroupBy` operator. With LINQ providers that don't treat their lambdas as expressions (e.g., LINQ to Objects), you could use a tuple instead, which would be slightly more succinct while having the same effect.

Conversion

Sometimes you will need to convert a query of one type to some other type. For example, you might have ended up with a collection where the type argument specifies some base type (e.g., `object`), but you have good reason to believe that the collection actually contains items of some more specific type (e.g., `Course`). When dealing with individual objects, you can just use the C# cast syntax to convert the reference to the type you believe you're dealing with. Unfortunately, this doesn't work for types such as `IEnumerable<T>` or `IQueryable<T>`.

Although covariance means that an `IEnumerable<Course>` is implicitly convertible to an `IEnumerable<object>`, you cannot convert in the other direction even with an explicit downcast. If you have a reference of type `IEnumerable<object>`, attempting to cast that to `IEnumerable<Course>` will succeed only if the object implements `IEnumerable<Course>`. It's quite possible to end up with a sequence that consists entirely of `Course` objects but that does not implement `IEnumerable<Course>`. Example 10-56 creates just such a sequence, and it will throw an exception when it tries to cast to `IEnumerable<Course>`.

Example 10-56. How not to cast a sequence

```
IEnumerable<object> sequence = Course.Catalog.Select(c => (object) c);
var courseSequence = (IEnumerable<Course>)sequence; // InvalidCastException
```

This is a contrived example, of course. I forced the creation of an `IEnumerable<object>` by casting the `Select` lambda's return type to `object`. However, it's easy enough to end up in this situation for real, in

only slightly more complex circumstances. Fortunately, there's an easy solution. You can use the `Cast<T>` operator, shown in Example 10-57.

Example 10-57. How to cast a sequence

```
IEnumerable<Course> courseSequence = sequence.Cast<Course>();
```

This returns a query that produces every item in its source in order, but it casts each item to the specified target type as it does so. This means that although the initial `Cast<T>` might succeed, it's possible that you'll get an `InvalidCastException` some point later when you try to extract values from the sequence. After all, in general, the only way the `Cast<T>` operator can verify that the sequence you've given it really does only ever produce values of type `T` is to extract all those values and attempt to cast them. It can't evaluate the whole sequence up front because you might have supplied an infinite sequence. If the first billion items your sequence produces will be of the right type, but after that you return one of an incompatible type, the only way `Cast<T>` can discover this is to try casting items one at a time.

TIP

`Cast<T>` and `OfType<T>` look similar, and developers sometimes use one when they should have used the other (usually because they didn't know both existed). `OfType<T>` does almost the same thing as `Cast<T>`, but it silently filters out any items of the wrong type instead of throwing an exception. If you expect and want to ignore items of the wrong type, use `OfType<T>`. If you do not expect items of the wrong type to be present at all, use `Cast<T>`, because if you turn out to be wrong, it will let you know by throwing an exception, reducing the risk of allowing a potential bug to remain hidden.

LINQ to Objects defines an `AsEnumerable<T>` operator. This just returns the source without modification—it has no effect at runtime. Its purpose is to force the use of LINQ to Objects even if you are dealing with something that might have been handled by a different LINQ provider. For example, suppose you have something that implements `IQueryable<T>`. That interface derives from `IEnumerable<T>`, but the extension methods

that work with `IQueryable<T>` will take precedence over the LINQ to Objects ones. If your intention is to execute a particular query on a database, and then use further client-side processing of the results with LINQ to Objects, you can use `AsEnumerable<T>` to draw a line that says, "This is where we move things to the client side."

Conversely, there's also `AsQueryable<T>`. This is designed to be used in scenarios where you have a variable of static type `IEnumerable<T>` that you believe might contain a reference to an object that also implements `IQueryable<T>`, and you want to ensure that any queries you create use that instead of LINQ to Objects. If you use this operator on a source that does not in fact implement `IQueryable<T>`, it returns a wrapper that implements `IQueryable<T>` but uses LINQ to Objects under the covers.

Yet another operator for selecting a different flavor of LINQ is `AsParallel`. This returns a `ParallelQuery<T>`, which lets you build queries to be executed by Parallel LINQ, a LINQ provider that can execute certain operations in parallel to improve performance when multiple CPU cores are available.

There are some operators that convert the query to other types and also have the effect of executing the query immediately rather than building a new query chained off the back of the previous one. `ToArray`, `ToList`, and `ToHashSet` return an array, list, or hash set, respectively, containing the complete results of executing the input query. `ToDictionary` and `ToLookup` do the same, but rather than producing a straightforward list of the items, they both produce results that support associative lookup. `ToDictionary` returns a `Dictionary<TKey, TValue>`, so it is intended for scenarios where a key corresponds to exactly one value. `ToLookup` is designed for scenarios where a key may be associated with multiple values, so it returns a different type, `ILookup<TKey, TValue>`.

I did not mention this lookup interface in Chapter 5 because it is specific to LINQ. It is essentially the same as a read-only dictionary interface, except the indexer returns an `IEnumerable<TValue>` instead of a single `TValue`.

While the array and list conversions take no arguments, the dictionary and lookup conversions need to be told what value to use as the key for each source item. You tell them by passing a lambda, as Example 10-58 shows. This uses the course's `Category` property as the key.

Example 10-58. Creating a lookup

```
ILookup<string, Course> categoryLookup =
    Course.Catalog.ToLookup(course => course.Category);
foreach (Course c in categoryLookup["MAT"])
{
    Console.WriteLine(c.Title);
}
```

The `ToDictionary` operator offers an overload that takes the same argument but returns a dictionary instead of a lookup. It would throw an exception if you called it in the same way that I called `ToLookup` in Example 10-58, because multiple course objects share categories, so they would map to the same key. `ToDictionary` requires each object to have a unique key. To produce a dictionary from the course catalog, you'd either need to group the data by category first and have each dictionary entry refer to an entire group or need a lambda that returned a composite key based on both the course category and number, because that combination is unique to a course.

Both operators also offer an overload that takes a pair of lambdas—one that extracts the key and a second that chooses what to use as the corresponding value (you are not obliged to use the source item as the value). Finally, there are overloads that also take an `IEqualityComparer<T>`.

You've now seen the most important standard LINQ operators, but since that has taken quite a few pages, you may find it useful to have a concise summary. Table 10-1 lists the operators and describes briefly what each is for. For completeness, this includes some additional less widely used operators.

Table 10-1. Summary of LINQ operators

Operator	Purpose
`Aggregate`	Combines all items through a user-supplied function to produce a single result.
`All`	Returns `true` if the predicate supplied is false for no items.
`Any`	Returns `true` if the predicate supplied is true for at least one item.
`Append`	Returns a sequence with all the items from its input sequence with one item added to the end.
`AsEnumerable`	Returns the sequence as an `IEnumerable<T>`. (Useful for forcing use of LINQ to Objects.)
`AsParallel`	Returns a `ParallelQuery<T>` for parallel query execution.
`AsQueryable`	Ensures use of `IQueryable<T>` handling where available.
`Average`	Calculates the arithmetic mean of the items.
`Cast`	Casts each item in the sequence to the specified type.
`Chunk`	Splits a sequence into equal-sized batches.
`Concat`	Forms a sequence by concatenating two sequences.
`Contains`	Returns `true` if the specified item is in the sequence.
`Count`, `LongCount`	Return the number of items in the sequence.
`DefaultIfEmpty`	Produces the source sequence's elements, unless there are none, in which case it produces a single element with a default value.
`Distinct`	Removes duplicate values.
`DistinctBy`	Removes values for which a projection produces duplicate values.

Operator	Purpose
`ElementAt`	Returns the element at the specified position (throwing an exception if out of range).
`ElementAtOrDefault`	Returns the element at the specified position (producing the element type's default value if out of range).
`Except`	Filters out items that are in the other collection provided.
`First`	Returns the first item, throwing an exception if there are no items.
`FirstOrDefault`	Returns the first item, or a default value if there are no items.
`GroupBy`	Gathers items into groups.
`GroupJoin`	Groups items in another sequence by how they relate to items in the input sequence.
`Intersect`	Filters out items that are not in the other collection provided.
`IntersectBy`	Same as `Intersect` but using a projection for comparison.
`Join`	Produces an item for each matching pair of items from the two input sequences.
`Last`	Returns the final item, throwing an exception if there are no items.
`LastOrDefault`	Returns the final item, or a default value if there are no items.
`Max`	Returns the highest value.
`MaxBy`	Returns the item for which a projection produces the highest value.
`Min`	Returns the lowest value.
`MinBy`	Returns the item for which a projection produces the lowest value.

Operator	Purpose
`OfType`	Filters out items that are not of the specified type.
`Order`	Produces items in an ascending order based on the value of the items themselves.
`OrderBy`	Produces items in an ascending order based on the value selected by a projection.
`OrderDescending`	Produces items in a descending order based on the value of the items themselves.
`OrderByDescending`	Produces items in a descending order based on the value selected by a projection.
`Prepend`	Returns a sequence starting with a specified single item, followed by all the items from its input sequence.
`Reverse`	Produces items in the opposite order than the input.
`Select`	Projects each item through a function.
`SelectMany`	Combines multiple collections into one.
`SequenceEqual`	Returns `true` only if all items are equal to those in the other sequence provided.
`Single`	Returns the only item, throwing an exception if there are no items or more than one item.
`SingleOrDefault`	Returns the only item, or a default value if there are no items; throws an exception if there is more than one item.
`Skip`	Filters out the specified number of items from the start.
`SkipLast`	Filters out the specified number of items from the end.
`SkipWhile`	Filters out items from the start for as long as the items match a predicate.
`Sum`	Returns the result of adding all the items together.
`Take`	Produces the specified number or range of items, discarding the rest.

Operator	Purpose
`TakeLast`	Produces the specified number of items from the end of the input (discarding all items before that).
`TakeWhile`	Produces items as long as they match a predicate, discarding the rest of the sequence as soon as one fails to match.
`ToArray`	Returns an array containing all of the items.
`ToDictionary`	Returns a dictionary containing all of the items.
`ToHashSet`	Returns a `HashSet<T>` containing all of the items.
`ToList`	Returns a `List<T>` containing all of the items.
`ToLookup`	Returns a multivalue associative lookup containing all of the items.
`Union`	Produces all items that are in either or both of the inputs.
`UnionBy`	Same as `Union` but using a projection for comparison.
`Where`	Filters out items that do not match the predicate provided.
`Zip`	Combines items at the same position from two or three inputs.

Sequence Generation

The `Enumerable` class defines the extension methods for `IEnumerable<T>` that constitute LINQ to Objects. It also offers a few additional (nonextension) static methods that can be used to create new sequences. `Enumerable.Range` takes two `int` arguments and returns an `IEnumerable<int>` that produces a sequentially increasing series of numbers, starting from the value of the first argument and containing as many numbers as the second argument. For example, `Enumerable.Range(15, 10)` produces a sequence containing the numbers 15 to 24 (inclusive).

`Enumerable.Repeat<T>` takes a value of type `T` and a count. It returns a sequence that will produce that value the specified number of times.

`Enumerable.Empty<T>` returns an `IEnumerable<T>` that contains no elements. This may not sound very useful, because there's a much less verbose alternative. You could write `new T[0]`, which creates an array that contains no elements. (Arrays of type `T` implement `IEnumerable<T>`.) However, the advantage of `Enumerable.Empty<T>` is that for any given `T`, it returns the same instance every time. This means that if for any reason you end up needing an empty sequence repeatedly in a loop that executes many iterations, `Enumerable.Empty<T>` is more efficient, because it puts less pressure on the GC.

Other LINQ Implementations

Most of the examples I've shown in this chapter have used LINQ to Objects, except for a handful that have referred to EF Core. In this final section, I will provide a quick description of some other LINQ-based technologies. This is not a comprehensive list, because anyone can write a LINQ provider.

Entity Framework Core

The database examples I have shown have used the LINQ provider that is part of Entity Framework Core (EF Core). EF Core is a data access technology that ships in a NuGet package, `Microsoft.EntityFrameworkCore`. (EF Core's predecessor, the Entity Framework, is still built into .NET Framework but is not in newer versions of .NET.) EF Core can map between a database and an object layer. It supports multiple database vendors.

EF Core relies on `IQueryable<T>`. For each persistent entity type in a data model, the EF can provide an object that implements `IQueryable<T>` and that can be used as the starting point for building queries to retrieve entities of that type and of related types. Since `IQueryable<T>` is not unique to the EF, you will be using the standard set of extension methods provided by the `Queryable` class in the

`System.Linq` namespace, but that mechanism is designed to allow each provider to plug in its own behavior.

Because `IQueryable<T>` defines the LINQ operators as methods that accept `Expression<T>` arguments and not plain delegate types, any expressions you write in either query expressions or as lambda arguments to the underlying operator methods will turn into compiler-generated code that creates a tree of objects representing the structure of the expression. The EF relies on this to be able to generate database queries that fetch the data you require. This means that you are obliged to use lambdas; unlike with LINQ to Objects, you cannot use anonymous methods or delegates with an EF query.

WARNING

Because `IQueryable<T>` derives from `IEnumerable<T>`, it's possible to use LINQ to Objects operators on any EF source. You can do this explicitly with the `AsEnumerable<T>` operator, but it could also happen accidentally if you used an overload that's supported by LINQ to Objects and not `IQueryable<T>`. For example, if you attempt to use a delegate instead of a lambda as, say, the predicate for the `Where` operator, this will fall back to LINQ to Objects. The upshot here is that EF will end up downloading the entire contents of the table and then evaluating the `Where` operator on the client side. This is unlikely to be a good idea.

Parallel LINQ (PLINQ)

Parallel LINQ is similar to LINQ to Objects in that it is based on objects and delegates rather than expression trees and query translation. But when you start asking for results from a query, it will use multithreaded evaluation where possible, using the thread pool to try to use the available CPU resources fully and efficiently. Chapter 16 will show multithreading in action.

LINQ to XML

LINQ to XML is not a LINQ provider. I'm mentioning it here because its name makes it sound like one. It's really an API for creating and parsing

XML documents. It's called *LINQ to XML* because it was designed to make it easy to execute LINQ queries against XML documents, but it achieves this by presenting XML documents through a .NET object model. The runtime libraries provide two separate APIs that do this: as well as LINQ to XML, it also offers the XML Document Object Model (DOM). The DOM is based on a platform-independent standard, and thus, it's not a brilliant match for .NET idioms and feels unnecessarily quirky compared with most of the runtime libraries. LINQ to XML was designed purely for .NET, so it integrates better with normal C# techniques. This includes working well with LINQ, which it does by providing methods that extract features from the document in terms of `IEnumerable<T>`. This enables it to defer to LINQ to Objects to define and execute the queries.

IAsyncEnumerable<T>

As Chapter 5 described, .NET defines the `IAsyncEnumerable<T>` interface, which is an asynchronous equivalent to `IEnumerable<T>`. Chapter 17 will describe the language features that enable you to use this. A full set of LINQ operators is available, although they are not built into the .NET runtime libraries. They are available in a NuGet package called `System.Linq.Async`.

Reactive Extensions

The Reactive Extensions for .NET (or Rx, as they're often abbreviated) are the subject of the next chapter, so I won't say too much about them here, but they are a good illustration of how LINQ operators can work on a variety of types. Rx inverts the model shown in this chapter where we ask a query for items once we're good and ready. So, instead of writing a `foreach` loop that iterates over a query, or calling one of the operators that evaluates the query such as `ToArray` or `SingleOrDefault`, an Rx source calls us when it's ready to supply data.

Despite this inversion, there is a LINQ provider for Rx that supports most of the standard LINQ operators.

Summary

In this chapter, I showed the query syntax that supports some of the most commonly used LINQ features. This lets us write queries in C# that resemble database queries but can query any LINQ provider, including LINQ to Objects, which lets us run queries against our object models. I showed the standard LINQ operators for querying, all of which are available with LINQ to Objects, and most of which are available with database providers. I also provided a quick roundup of some of the common LINQ providers for .NET applications.

The last provider I mentioned was Rx. But before we look at Rx's LINQ provider, the next chapter will begin by looking at how Rx itself works.

[1] If you do so, be careful not to confuse it with another WPF type, `Rectangle`. That's an altogether more complex beast that supports animation, styling, layout, user input, databinding, and various other WPF features. Do not attempt to use `Rectangle` outside of a WPF application.